

Programmation des Architectures Parallèles

Rapport étape 3 : projet-06

20 avril 2026

1. — Introduction et contexte de l'étape 3

1.1 — Objectifs de l'étape 3

L'étape 2 nous a permis d'éliminer le calcul des zones stables grâce à l'évaluation paresseuse et au parallélisme OpenMP sur les architectures hybrides P-Cores/E-Cores. L'étape 3 ouvre deux nouvelles perspectives. D'une part la vectorisation utilisant **AVX**, qui consiste à exploiter le parallélisme *intra-registre* du CPU en traitant simultanément huit entiers 32 bits par instruction. D'autre part, le portage via **OpenCL** permettant de tirer parti des milliers de cœurs du GPU, dont la hiérarchie mémoire (registres, mémoire locale, mémoire globale) est différente de celle du CPU.

Ces deux approches s'appliquent au noyau synchrone (*ssandPile*), pour lequel la version *lazy* de l'étape précédente constitue la référence séquentielle de comparaison. Le cas asynchrone n'est quant à lui étudié qu'à travers le spectre de la vectorisation dans cette étape.

1.2 - Configuration matérielle et contraintes GPU

Afin d'évaluer les gains de performance apportés par nos optimisations GPU et paresseuses, nos expérimentations ont été menées sur les machines de la salle 104, dont les caractéristiques sont les suivantes.

Premièrement, et comme souligné dans nos précédents travaux, le processeur hôte possède une architecture hybride composée de 8 P-cores et 16 E-cores supportant le jeu d'instructions vectoriels AVX2 (256 bits). Les P-cores sont conçus pour les calculs intensifs tandis que les E-cores, regroupés en clusters, privilégient l'efficacité énergétique en partageant leur cache L2 (Une description détaillée des caches est en annexe D). Lors de l'exécution OpenCL, ce processeur agit comme l'*Host*, pilotant l'exécution des noyaux et gérant les transferts de données.

De plus, cette étape repose en partie sur l'exploitation de l'accélérateur graphique. La carte NVIDIA RTX 4000 Ada utilisée s'appuie sur l'architecture Ada Lovelace et offre :

- 48 Streaming Multiprocessors, regroupant un total de 6144 cœurs CUDA ;
- 20 Go de mémoire globale (GDDR6) avec une bande passante théorique de l'ordre de 360 Go/s ;
- jusqu'à 128 Ko par SM, configurable, servant de cache programmable (`__local`) à très faible latence.

Il faut noter que l'optimisation d'un noyau OpenCL *sandpile* nécessite de composer avec la structure hiérarchique illustrée dans la figure 1. La latence élevée de la mémoire globale impose de minimiser les accès en VRAM et de privilégier la coalescence. L'unité d'exécution de base, le *warp* (32 threads en NVIDIA), exécute la même instruction en *lockstep* ; toute divergence de branchement entraîne une sérialisation coûteuse.

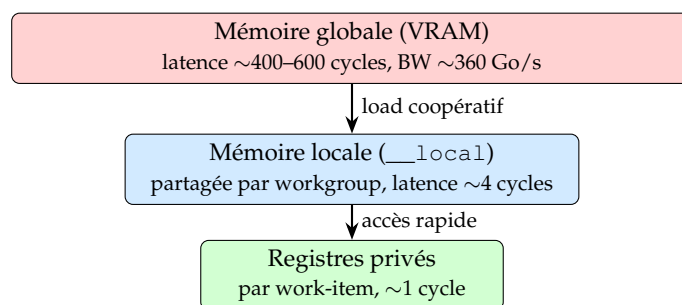


FIGURE 1 – Hiérarchie mémoire GPU : l'objectif des optimisations (`opt3`) est de maximiser la proportion des accès servis par la mémoire locale et les registres en minimisant les allers-retours avec la VRAM.

2. — Vectorisation AVX2 (ssandPile et asandPile)

2.1 — Principes de la vectorisation explicite

La vectorisation du traitement des tuiles se fait sous AVX2. Les vecteurs utilisés sont des vecteurs de 256 bits, en l'occurrence, cela revient à traiter 8 entiers de 32 bits. La vectorisation du noyau synchrone se fait différemment du noyau asynchrone. Dans le cas du noyau synchrone, la vectorisation est triviale. Dans le cas asynchrone, on doit rendre le noyau synchrone localement au moyen d'un buffer, puis stocker le résultat après avoir fini le calcul.

Nous comparerons les différentes versions AVX entre-elles et par rapport à la version classique des deux kernels. Par ailleurs, nous ne considérerons dans cette partie que le gain obtenu par la vectorisation elle-même, soit en séquentiel. Lors de la comparaison à la version non AVX, nous utiliserons comme référence les performances de la fonction de tuile "opt" réalisée lors de l'étape 1.

Les expériences réalisées porteront sur une version séquentielle, utilisant la fonction de calcul "tiled" (sans optimisation de type lazy), sur un problème plein de taille 1024. De plus, les tailles de tuiles seront de 32x32, et nous limiterons les calculs à 300 itérations.

Ces paramètres seront valables pour ssandPile et asandPile.

2.2 — Implémentation du noyau synchrone AVX2

Comme dit précédemment, la vectorisation du noyau synchrone est triviale. L'implémentation naïve que nous en avons faite consistait à traverser l'entière de la grille horizontalement. Il s'agit d'une version très simple à écrire, mais elle n'est pas très optimisée. Avec cette méthode de parcours, chaque tour de boucle nous fait charger 5 vecteurs, le vecteur de la cellule actuelle, le vecteur décalé à gauche, le vecteur décalé à droite, et les deux vecteurs verticaux. Étant donné que le chargement d'un vecteur est très coûteux, on préfère alors opérer autrement. La deuxième version de notre implémentation (nommée 'avx_opt') effectuait un parcours vertical de la tuile. Ce parcours permet, à chaque tour de boucle, de ne charger que 3 vecteurs. En effet, avec un tel parcours, des vecteurs peuvent être récupérés de l'itération précédente : le vecteur supérieur provient du vecteur central, et le vecteur central provient du vecteur inférieur.

On peut comparer les performances des deux implémentations avec notre fonction opt (Figure 2).

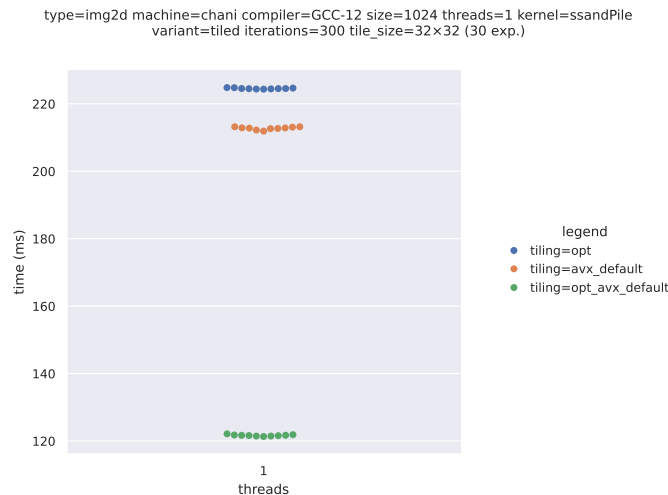


FIGURE 2 – Différence de temps parcours latéral(avx)/vertical(opt_avx)

Note importante, les fonctions de tuilages sont nommées sous deux versions, une classique, et une "default", default signifie que les bords ne sont pas traités, nous considérerons principalement des versions default dans un premier temps, et nous parlerons du traitement des bords plus tard.

En observant les résultats de notre expérience, on peut remarquer qu'en effet, le traitement vertical est bien meilleur. On pouvait se douter de cela : Le seul inconvénient du traitement vertical serait les cache miss. Cependant, le coût du load de 2 vecteurs en plus écrase ce désavantage. Par ailleurs, il se peut que le nombre de cache miss soit le même pour les deux parcours D. Sous cette hypothèse, le parcours vertical est strictement meilleur que le parcours horizontal.

2.3 — Implémentation du noyau asynchrone AVX2

Le noyau asynchrone, nécessite, pour sa réalisation, l'utilisation d'une sous-tuile qui sera traitée de façon synchrone dans notre problème. Nous allons donc utiliser un tableau de vecteurs. Pour se faire nous effectuerons le

même parcours que pour le noyau synchrone, soit en colonnes. La raison derrière ce choix est l'économie de vecteurs à utiliser pour faire une itération. Dans notre cas, nous devons stocker 2 vecteurs en plus, les vecteurs haut et en bas de la sous-tuile. Par soucis de simplicité, nous utiliserons un deuxième tableau de vecteur dont les éléments à gauche et à droite posséderont les valeurs envoyées à gauche et à droite de notre sous-tuile. Nous avons aussi ajouté un système d'itérations internes sur plusieurs niveaux au début, sur la tuile entière et sur les sous-tuiles. Il s'est avéré après plusieurs tests que les itérations de la tuile entière n'apportaient que peu ou même heurtaient les performances. Nous nous sommes donc limités aux itérations internes. Par ailleurs, nous avons gardé une constante de 2 itérations pour ce nombre, cette valeur ne sera pas abordée dans les expérimentations, nous la mentionnons donc simplement.

Pour calculer combien chaque élément du vecteur reçoit de ses voisins, nous avons besoin de décaler sur la gauche/la droite l'entièreté du vecteur. Cela peut se faire en utilisant les fonctions de rotation des éléments données, mais ces dernières ne sont pas les plus optimisées pour résoudre ce problème. Les vecteurs de 256 bits en AVX sont partagés sous forme de lanes de 128 bits. Les opérations intra lane sont facile à exécuter tandis que celles les mélangeant (comme `_mm256_permutevar8x32_epi32`) sont coûteuses. On se propose de regarder le resultat voulu dans notre cas (Figure 3).

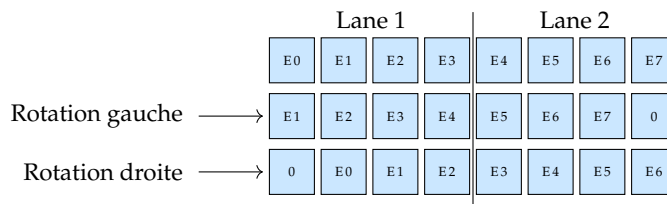


FIGURE 3 – Visualisation des lanes AVX lors des rotations naïves : E3 et E4 traversent la frontière inter-lane.

On remarque que les seuls éléments à traverser une lane sont les éléments E3 et E4. Ainsi, on se propose d'effectuer le protocole suivant : On va d'un seul coup échanger E3 et E4 dans un vecteur au moyen d'un unique `_mm256_permutevar8x32_epi32`. Puis, nous allons utiliser les intrinsèques `_mm256_srli_epi32` (droite) et `_mm256_slli_epi32` (gauche) pour effectuer un shift intra lane (Figure 4) :

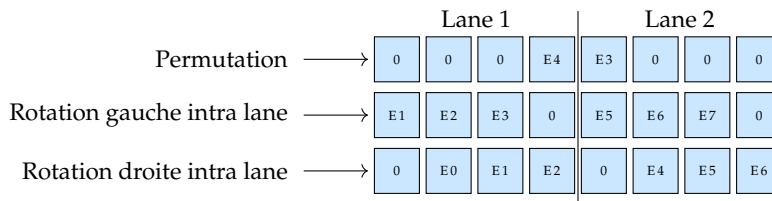


FIGURE 4 – Visualisation des lanes AVX avec permutation préalable de E3/E4 suivie de shifts intra-lane.

Il ne reste alors qu'à placer les éléments au moyen d'un blend. On se propose comme avant de visualiser les performances comparatives sur une exécution séquentielle (Figure 5) :



FIGURE 5 – Différence de temps AVX avec rotation et sans rotation

Le tuillage "default" correspond à notre implémentation utilisant les `shift intra lane`, la version `rotate` utilise les fonctions de rotation habituelles. Nos deux implémentations sont meilleures que la version `opt` de base, et on remarque que notre version avec `shift` est légèrement meilleure. A noter que la différence est visible malgré que cette partie du calcul ne représente que 2 opérations dans notre boucle d'itérations sur un vecteur. Dans le cas où l'on utilisait des `shift` de façon plus intensive, on pourrait sans doute voir un bien meilleur résultat.

2.4 — Stratégie de traitement des bords

Le traitement des bords est similaire pour `ssandPile` et `asandpile`. La différence majeure est que `ssandpile` ne différencie pas les bords verticaux. Pour expliquer les deux traitements, nous considérerons la grille représentant les cases du problème 6. Pour le cas `assandpile` et `ssandpile`, on utilisera un `mask`, un vecteur classique contenant des -1 sauf sur la case de gauche et la case de droite, qui elles dépendent de si l'on est sur un bord droit ou gauche (pour chaque côté, 0 si on est sur un bord, -1 autrement).

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

FIGURE 6 – Exemple de grille d'éléments (cases de bord en rouge, cases intérieures en vert).

2.4.1 — Traitement des bords pour `ssandpile`

Le problème principal du bord gauche et droit est que l'un des vecteurs de notre traitement devient un vecteur "incomplet", soit décalé à droite ou à gauche, avec un élément de moins. Nous souhaitons éviter au maximum le nombre de `maskstore/maskload`, et simplement d'opérations masquées que nous effectuons. Pour réaliser cela, nous allons considérer les vecteurs de bords comme les vecteurs complets, soit en restaurant la largeur du traitement. Pour le bord gauche, on ne partira non pas de $x = 1$, mais bien de $x = 0$. En notant que, sachant que l'on ne traite pas la première et dernière ligne, l'élément directement à gauche(respectivement droite) de la première(respectivement dernière) case de notre ligne existe dans notre tableau. Par exemple dans notre grille, la case directement à gauche de la case 6 est la case 5, celle directement à droite de la case 10 est la case 11. On peut aussi remarquer que de telles cases sont aussi des cases de bords et peuvent être modifiées ou lues pour peu que l'on ignore complètement leurs valeurs.

A partir de là, on charge normalement les vecteurs au centre, à gauche, et à droite, avec potentiellement donc un overflow sur la dernière case de la ligne supérieure ou première case de la ligne inférieure. On traite alors normalement les valeurs. La vérification ne se fait que lors du store : on utilise le masque spécifié plus tôt pour décider si la valeur doit être sauvegardée ou non.

2.4.2 — Traitement des bords pour `asandpile`

Le traitement des bords horizontaux est identique pour `asandpile` que celui de `ssandpile`, la seule différence étant que le `mask` est utilisé lors du load des valeurs et non pas lors du store. Cependant, notre version asynchrone impose une vérification pour les bords verticaux. Comme dit précédemment, nous considérons un tableau d'une taille donnée (appelée `SUBTILE_SIZE`) avec deux éléments supplémentaires pour gérer nos sous-tuiles, en supposant que `SUBTILE_SIZE` est un diviseur de la taille de tuile. Dans le cas d'un bord supérieur ou inférieur, la taille de la tuile change et détruit cette hypothèse.

Nous décomposons alors notre calcul. Premièrement, nous allons utiliser la même logique que pour le bord gauche et considérer $y = 0$ si $y = 1$. Plutôt que d'utiliser `SUBTILE_SIZE` directement, nous utiliserons la variable `effective_subtile_size`. Cette variable est strictement équivalente à `SUBTILE_SIZE`, et on lui retranche 1 si l'on est en train de traiter la sous-tuile correspondant au bord haut ou bas. Désormais, le `mask` s'applique lors du chargement et non pas lors du store pour éviter les cases incorrectes.

2.4.2 — Impacts sur les performances

Pour rappel, nous considérons ici uniquement les performances relatives des implémentations AVX selon leur traitement des bords.

Ici 7, la version `opt_avx_sides` correspond à la version où l'on ne traite que les bords latéraux. A noter que, surprenamment, les performances sont bien pires pour les versions avec traitement des bords pour `asandPile`. Cela peut venir de 2 choses, soit une mauvaise implémentation de notre part, soit que nos idées de traitement ne sont pas adaptées au problème des bords pour la version asynchrone. Par exemple, le nombre de calculs supplémentaires nécessaire au traitement des bords peut être trop grand par rapport à l'apport du traitement des bords en lui-même.

type=img2d machine=joe compiler=GCC-12 size=1024 threads=1 variant=tilted iterations=300
tile_size=32x32 (70 exp.)

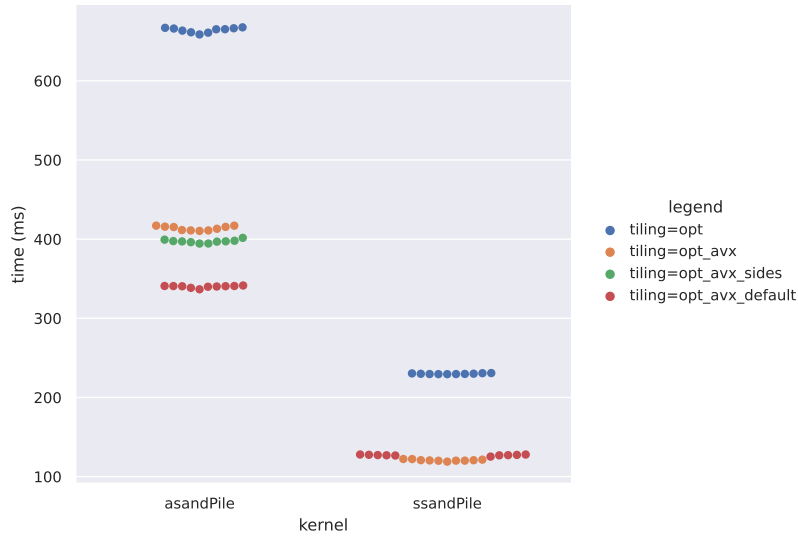


FIGURE 7 – Comparaison des versions AVX ssandpile/asandpile en opt et default (avec et sans traitement des bords 1024×1024).

Nous n’avons pas plus cherché pour cette partie là et avons simplement considéré la version default pour nos futurs tests sur asandPile. Enfin, nous n’explorerons pas ici les tailles de tuiles, car elles seront en partie explorées dans nos tests spécifiques ssandpile.

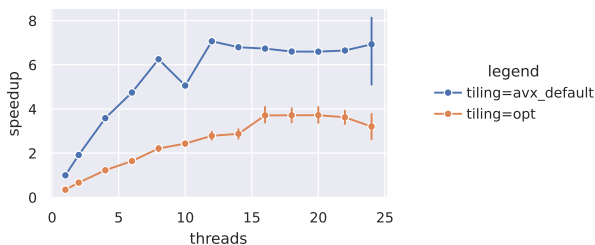
3. — AVX parallèle : (ssandPile et asandPile)

Le focus de ce rapport était beaucoup plus sur l’explication de nos choix et nos résultats quant à l’implémentation AVX en elle-même, ainsi, nous ne nous focaliserons pas (principalement par manque d’espace) sur l’entièreté des tests et faisons le choix de n’en parler que de 2, le cas 512 pour asandPile, et le cas 2048 pour ssandPile, par ailleurs, étant donné la proximité des paramètres avec l’étape précédente, nous ne parlerons pas de scheduling sur nos cas. Le principal objectif ici sera de faire le lien avec les gains de performances par rapport aux versions classiques et d’expliquer certaines limites de l’AVX dans nos cas.

3.1— Cas 512 plein

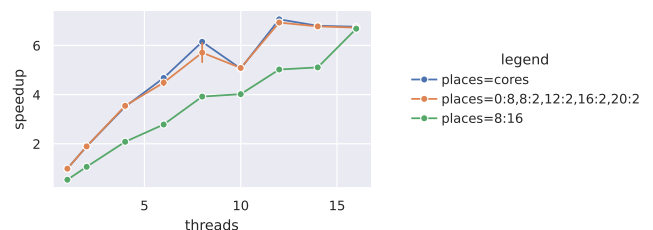
Pour la gestion du cas 512 plein, nous allons d’abord nous intéresser à la courbe de speedup selon le nombre de threads avec des paramètres similaires au 256 de l’étape précédente. En particulier, nous utilisons toujours la 2-coloration, soit chaque thread traite des lignes de tuiles plutôt que des tuiles entières.

type=img2d machine=shaddam compiler=GCC-12 size=512 kernel=asandPile
variant=omp_2coloring iterations=5000 places=cores refTime=700
tile_size=8x16 (130 exp.)



(a) Speedup total.

type=img2d machine=hawat compiler=GCC-12 size=512 kernel=asandPile
variant=omp_2coloring tiling=avx_default iterations=2000 schedule=static refTime=285
tile_size=8x16 (135 exp.)



(b) Speedup selon la répartition des coeurs.

FIGURE 8 – Comparaison des traces d’exécution avant (gauche) et après (droite) la vectorisation de la fonction de regroupement des bitsets.

Dans notre courbe de speedup totale 8a, on remarque que le speedup stagne après environ 12 threads. Ce dernier devient par ailleurs beaucoup moins stable à 24 threads. Plusieurs choses peuvent expliquer cela. La première est une

mauvaise répartition du travail, la deuxième est une baisse des performances des coeurs à mesure que l'on demande une plus grande charge de travail vectorisée. Pour rappel, des coeurs utilisant l'AVX font baisser la fréquence globale des coeurs. Ainsi, si on observe la répartition, on remarque qu'à 12 coeurs la répartition du travail est la suivante : 8 P-cores et 4 E-cores. Il semblerait que cette répartition soit celle qui partage le mieux la charge sans pour autant impacter sur la vitesse des coeurs. En effet, après cela, on peut supposer que le nombre d'E-cores devient trop grand par rapport aux P-cores et que ces derniers sont moins affectés par la baisse de fréquence. On peut alors se demander quel serait le résultat si l'on utilisait uniquement des E-cores. Par ailleurs, il serait aussi intéressant de savoir si la répartition des E-cores en groupes peut-être exploitée ou non.

On peut voir 8b que l'utilisation des E-cores seulement rend bien les performances moins bonnes comme attendu. Cependant on voit un regroupement aux alentours de 16 threads. Cela est assez surprenant et peut supposer plusieurs choses. Soit le fait que le partage de coeurs soit égal pour la courbe de P-cores et E-cores affecte trop les performances ou potentiellement les E-cores sont en réalités moins affectés si il n'y a pas de P-cores utilisant AVX, d'autant que la courbe n'a jusque là fait que monter. Nous n'avons pas pu faire d'expériences répondant vraiment à ces questions malheureusement. On remarque ensuite que la répartition des E-cores en groupes, soit le plus espacé possible, n'a pas vraiment d'impact positif sur les performances. On peut en déduire que la baisse de fréquence est la même pour chaque coeur est n'en impacte pas certains plus que d'autres (sauf pour la différence P-core/E-core).

3.2— Cas 2048 spirals

Nous en avons parlé plus tôt, mais pour le cas synchrone, il y a une certaine contrepartie dans notre implémentation AVX. En effet, en parcourant la tuile en vertical, nous choisissons de sacrifier un bon parcours pour le cache contre moins de loads. Cependant la taille de tuile peut potentiellement atténuer ce phénomène. Nous allons donc nous concentrer sur ce point en premier lieux.

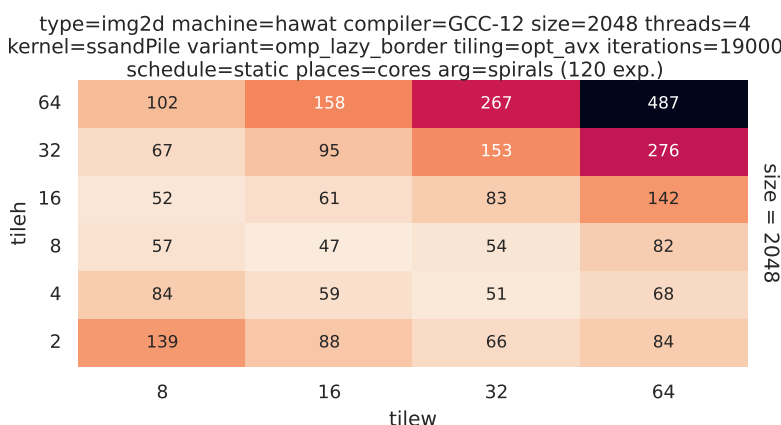
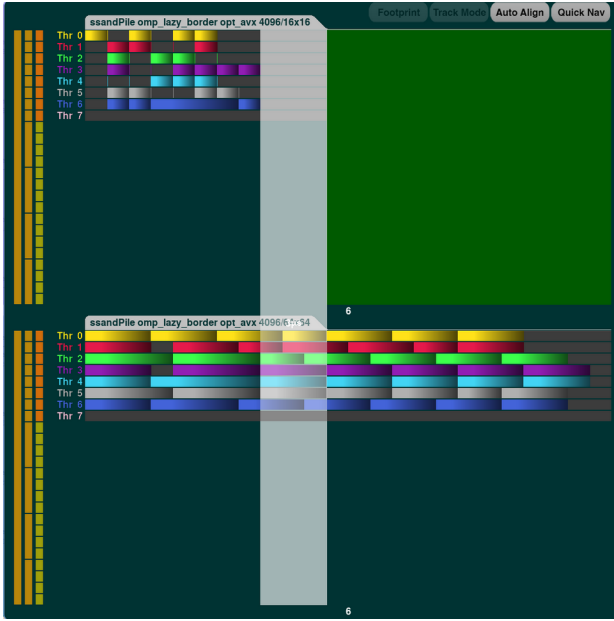


FIGURE 9 – Heatmap ssandPile 2048

On remarque que, de façons générale, tant que l'on reste sur une taille d'environ 16 * 8 9, ou leurs équivalents en changeant les coefficients, les performances restent bonnes. Nous remarquons cependant que rester en dessous de 8 donne les meilleures performances. Pour rappel, le parcours vertical provoque en règle général plus de cache miss que le parcours horizontal, on peut donc en déduire que rester en dessous de 8 (sans pour autant mettre une taille trop basse) semble limiter ces caches miss. Il est important de noter que cela ne sont que des hypothèses et qu'en réalité, étant donné l'évaluation lazy, il est difficile de savoir si cela correspond bien à un meilleur parcours des tuiles ou simplement à un meilleur découpage. En l'occurrence, la taille 16x8(hauteurxlargeur), bien que moins bonne que les autres, semble indiquer que la taille globale reste le plus gros facteur.

Durant nos tests, l'étude des autres paramètres sur le cas 2048 ne nous a pas tant apporté, car ces derniers étaient les mêmes que pour le cas 1024. Par ailleurs, cette simple étude des paramètres ne nous a pas permis d'obtenir tant de gains en performances, cela nous a poussé à nous demander si le problème ne venait pas d'ailleurs. Nous avons donc investigué. Il est bon de rappeler que les cas ssandPile creux utilisent le bitset que nous avons défini lors du dernier rendu comme structure de donnée lazy. Nous ne rappellerons pas son fonctionnement en détail ici. Nous rappellerons simplement que les tuiles sont représentées par les bits d'entiers (64 tuiles par entier), et que chaque thread écrit dans une version locale de son bitset, enfin un regroupement est fait à la fin de chaque itération. Nous voulons montrer un comportement que nous avons observé durant nos tests. Il peut-être révélé via une trace, ici sur un problème de taille 4096 pour le rendre plus visible. (Figure 10).



(a) Trace initiale : attente visible entre itérations pour petites tuiles.



(b) Trace après vectorisation du merge : réduction du temps d'attente.

FIGURE 10 – Comparaison des traces d'exécution avant (gauche) et après (droite) la vectorisation de la fonction de regroupement des bitsets.

On compare ici (Figure 10a) deux exécutions, l'une avec une petite taille de tuile, l'autre avec une très grande taille. Le premier cas implique beaucoup d'entier dans notre bitset, le deuxième beaucoup moins. A noter que nous ne comparons pas les performances des deux itérations. Ici, on remarque que la prochaine itération attend un long moment avant de continuer à la suivante pour le premier cas, alors que l'autre trace ne présente pas ce problème. Nous en avons déduit deux cas possibles : Soit les threads passent leur temps à vérifier si une tuile doit être faite sans que ce soit le cas, soit le problème provient du rassemblement des bitsets en un à la fin de l'itération. Malheureusement, nous ne pouvons pas agir sur le premier, mais nous pouvons essayer de régler le second. Lors du dernier rendu nous avons discuté de notre tentative de paralléliser le regroupement sans succès. Nous avons donc décidé de vectoriser la dite fonction de rassemblement. Son implémentation peut être trouvée en annexe. On compare alors la trace de la nouvelle version avec l'ancienne (Figure 10b). Notre nouvelle version est celle représentée en dessous. Ici, on remarque que pour une itération où les deux exécutions devraient se terminer en même temps, la nouvelle version du merge se termine légèrement en avance. Nous avons pris un cas extrême avec un grand nombre d'entier (1024), nous voulons donc savoir si notre optimisation serait visible pour un cas réel.

type=img2d machine=hawat compiler=GCC-12 size=2048 kernel=ssandPile tiling=opt_avx
iterations=33398 schedule=static places=cores arg=spirals refTime=204 tile_size=8x32 (130
exp.)

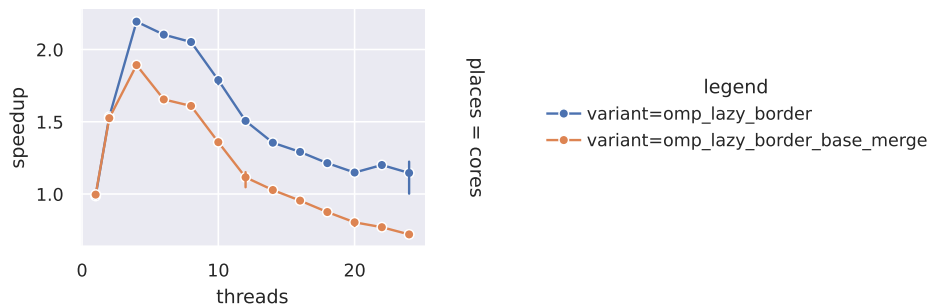


FIGURE 11 – Speedup pour merge avec et sans AVX

De retour sur notre cas 2048 11, nous voulons observer l'effet de notre merge sur le speedup, car en effet le nombre de threads devrait avoir un grand effet sur le coût du merge. Ici, la seule différence est la fonction de merge à la fin de chaque itération. On remarque que le speedup de notre merge vectorisé est bien visible. En vérité, cela

semble plutôt logique : Dans une configuration avec peu de calcul comme la configuration `spirals`, le nombre de calculs du problème est extrêmement faible pour une itération, autrement dit, on peut se douter que notre merge, qui lui a un coût constant peu importe la charge de travail, peut devenir un facteur non négligeable dans notre coût de calcul. Nous discutons d'un moyen de rendre ce coût constant peu-importe le nombre de threads en annexe D. Dans tous les cas, cela nous a fait prendre conscience de l'importance d'utiliser les mêmes méthodes d'optimisations sur nos structures de données que sur le problème en lui-même.

4. — Portage GPU : OpenCL (`ssandPile`)

4.1 — Architecture du programme OpenCL

Le programme OpenCL se décompose en deux parties. Côté *host* (CPU), une fonction `ssandPile_compute_ocl_*` orchestre les lancements de kernels, gère l'échange des tampons et détecte la convergence. Côté *device* (GPU), le kernel correspondant effectue le calcul proprement dit en mémoire locale. La figure 12 résume ce flot pour la version de référence.

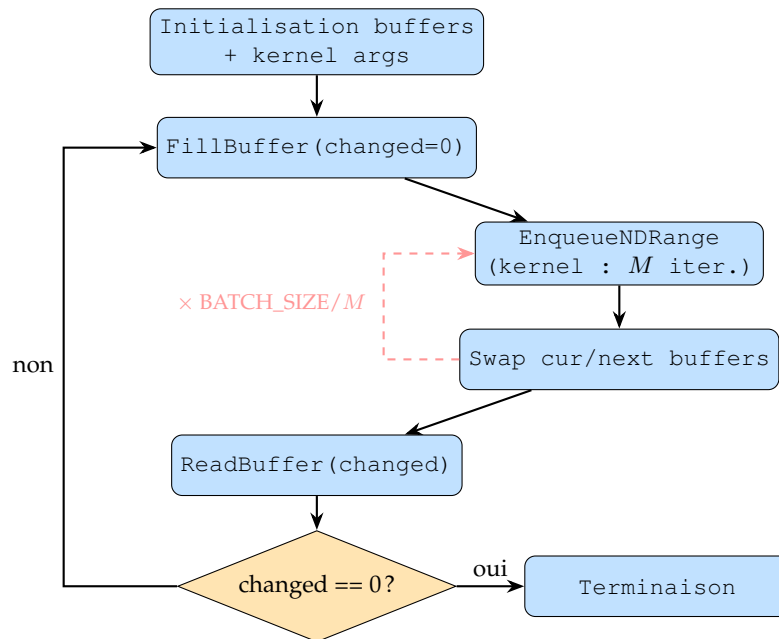


FIGURE 12 – Flot d'exécution côté *host*. Le transfert mémoire (`ReadBuffer`) est effectué une seule fois tous les `BATCH_SIZE` itérations pour amortir son coût. M désigne le nombre d'itérations par lancement de kernel (4, 5, 6 ou 8 selon la variante).

Nous avons développé plusieurs variantes du kernel, chacune augmentant progressivement le nombre d'itérations réalisées en mémoire locale par lancement. Le tableau 1 résume ces variantes. Néanmoins, nous discuterons seulement des trois premières au cours du rapport pour illustrer les optimisations intéressantes qui furent développées.

Variante (-v)	Marge	Passes	Particularité
<code>ocl</code>	1	—	naïve, 1 itération par lancement, accès globaux
<code>ocl_multi_m4_loop</code>	4	boucle	chargement coopératif par boucle, flexible sur la taille de workgroup
<code>ocl_multi_m5_static</code>	5	3-pass	chargement statique, lié à un workgroup 32×16
<code>ocl_multi_m6_robust</code>	6	3-pass	tail-handler pour un nombre exact d'itérations
<code>ocl_multi_m8_static</code>	8	4-pass	densité maximale, pression LDS élevée
<code>ocl_multi_m5_static_quad</code>	5	3-pass	calcul sur le quadrant seul (symétrie 4)

TABLE 1 – Variantes de kernels OpenCL implémentées. La marge correspond au nombre d'itérations réalisées en mémoire locale par lancement; les passes indiquent la stratégie de chargement de la tuile étendue.

4.2 — Version naïve et détection de terminaison

La version naïve (`ssandPile_ocl`) attribue un work-item à chaque cellule de la grille. Chaque work-item lit ses quatre voisins depuis la mémoire globale, calcule la nouvelle valeur et l'écrit dans le buffer de sortie. Cette version ne réalise qu'une seule itération par lancement de kernel.

La détection de convergence repose sur un drapeau atomique `changed` en mémoire globale. Pour en limiter le coût, nous utilisons une réduction locale par workgroup : chaque work-item positionne un flag en `__local`, une réduction parallèle arborescente réduit ce tableau à un seul entier, et seul le thread 0 du groupe effectue un `atomic_or` vers le drapeau global. Le host ne lit ce drapeau qu'une fois tous les `BATCH_SIZE` lancements.

4.3 — Mémoire locale et marge d'itérations

La principale optimisation repose sur le chargement d'une tuile *étendue* en mémoire locale, incluant une marge de `MARGIN` cellules de chaque côté. Cela permet de réaliser `MARGIN` itérations consécutives entièrement en mémoire locale, sans relire la mémoire globale, au prix d'un unique chargement initial. La figure 13 illustre ce principe.

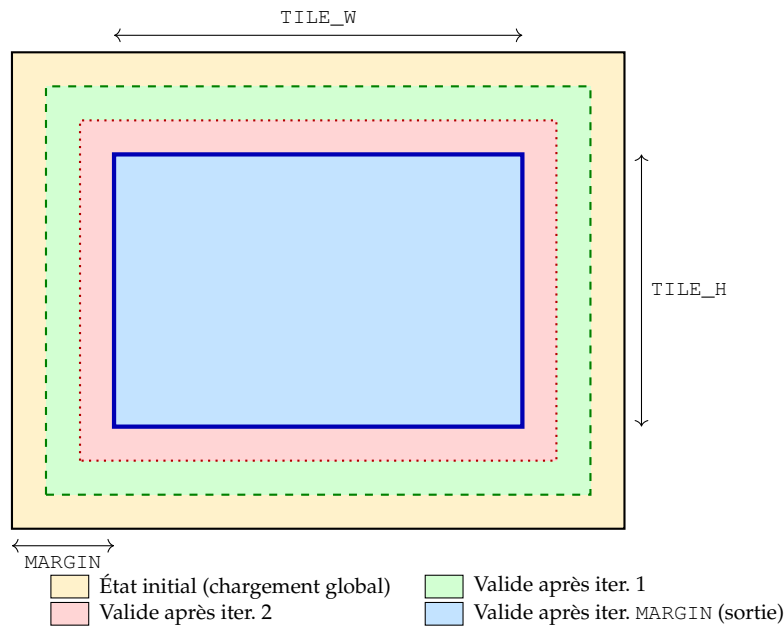


FIGURE 13 – Rétrécissement de la zone valide. Pour obtenir $TILE_W \times TILE_H$ cellules correctes après `MARGIN` itérations, on charge une zone plus large. Chaque itération « consomme » une couche de cellules sur les bords.

4.4 — Du chargement par boucle au chargement statique

La première version multi-itérations (`multi_m4_loop`, marge 4) utilise un chargement coopératif par boucle classique :

Listing 1 – Chargement coopératif par boucle (`multi_m4_loop`)

```
1 for (int i = tid; i < total_cells; i += wg_size) {
2     int tx = i % IN_W, ty = i / IN_W;
3     int g_x = clamp(base_gx - MARGIN + tx, 0, DIM - 1);
4     int g_y = clamp(base_gy - MARGIN + ty, 0, DIM - 1);
5     tile0[i] = in[g_y * DIM + g_x];
6 }
7 barrier(CLK_LOCAL_MEM_FENCE);
```

Cette approche est flexible et fonctionne pour toute taille de workgroup, mais engendre un surcoût de contrôle (calcul de `tx/ty` à chaque itération de boucle, branchement sur la borne). La version `multi_m5_static` supprime ce surcoût en remplaçant la boucle par un mapping statique à 3 passes. Chaque thread charge exactement 2 ou 3 cellules, à des indices précalculés à la compilation. Avec un workgroup de $32 \times 16 = 512$ threads et une tuile étendue de $43 \times 26 = 1118$ cellules, 3 passes ($3 \times 512 = 1536 \geq 1118$) suffisent.

De même, les itérations locales sont réalisées via une macro `PASS_OPT` qui applique le masque sur les mêmes 3 indices statiques, alternant entre `tile0` et `tile1` :

Listing 2 – Macro de passe statique pour les itérations en mémoire locale

```

1 #define PASS_OPT(step, t_in, t_out) \
2   if (tx1 >= step && tx1 < IN_W - step && ...) { \
3       t_out[i1] = (t_in[i1] & 3) \
4       + (t_in[i1 - IN_W] >> 2) + ...; \
5   } \
6   /* idem pour i2, i3 */ \
7   barrier(CLK_LOCAL_MEM_FENCE); \
8
9 PASS_OPT(1, tile0, tile1); // iter 1
10 PASS_OPT(2, tile1, tile0); // iter 2
11 // ...

```

À la dernière itération, seules les $TILE_W \times TILE_H$ cellules centrales sont écrites vers `out[]` en mémoire globale.

4.5 — Batching et amortissement de la détection de terminaison

La détection de terminaison nécessite un transfert mémoire bloquant du GPU vers le CPU (`clEnqueueReadBuffer` avec `CL_TRUE`), dont le coût est de l'ordre de quelques dizaines de microsecondes. Pour l'amortir, nous regroupons $BATCH_SIZE/M$ lancements de kernels (chacun réalisant M itérations internes) avant chaque lecture du drapeau `changed`.

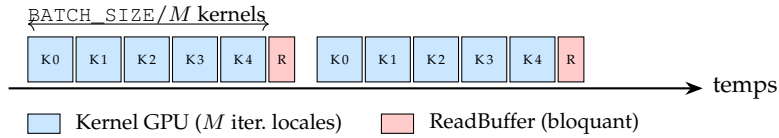


FIGURE 14 – Stratégie de batching : le transfert mémoire bloquant (R) est effectué une seule fois tous les $BATCH_SIZE$ itérations, réduisant son impact relatif sur le temps total.

5. — Expérimentations GPU

Dans cette section, nos expériences se concentreront sur la configuration initiale `4partout` avec une grille de taille 4096×4096 sur 1000 itérations. Pour illustrer de manière concise l'impact des différentes optimisations matérielles, nous nous focaliserons sur un sous-ensemble représentatif de nos noyaux de calcul.

5.1 — Comparaison des variantes de kernel

Nous comparons, dans la figure 15, trois implémentations OpenCL : la version de base effectuant un unique passage (`ocl`), la version utilisant une marge en mémoire locale avec des boucles dynamiques (`ocl_multi_m4_loop`), et notre version avancée à chargement statique (`ocl_multi_m5_static`).

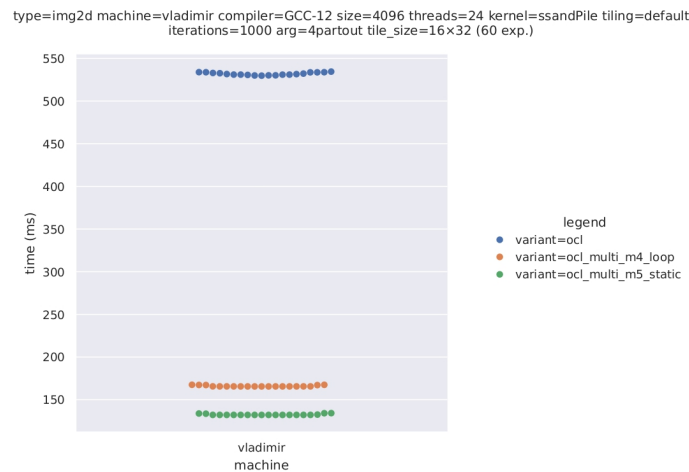


FIGURE 15 – Comparaison des temps d'exécution des noyaux OpenCL (`ssandPile`, configuration `4partout` 4096×4096 , tuiles de 32×16).

Le premier constat est l'écart de performance important entre la version naïve et les implémentations tuilées. Le noyau naïf effectue des requêtes de lecture répétées vers la mémoire globale (qui possède une très forte latence) pour chaque itération. L'ajout d'une marge (de taille 4 pour le second, et 5 pour le troisième) s'avère bénéfique car elle permet de charger une large zone en mémoire locale (la *Shared Memory* pour NVIDIA), qui est un espace prioritaire et à faible latence présent sur chaque *Streaming-Multiprocessor*. Ce passage en mémoire partagée permet de réaliser l'équivalent de 4 ou 5 itérations d'un coup. Le coût de lecture et d'écriture de chaque cellule vers la mémoire globale est ainsi largement amorti.

On observe ensuite une différence entre `m4_loop` et `m5_static`. Cette différence réside entre une gestion dynamique et statique du chargement. La version `m4_loop` se base sur des boucles avec d'importantes opérations de modulo à chaque tour et de vérification des bords. Ces éléments requièrent des unités arithmétiques, et peuvent créer des comportements variés entre les threads de calcul. Les GPU exécutent les instructions de manière synchrone par lots de 32 threads, structurés en blocs appelés *warps*. Si les threads au sein du même *warp* empruntent des chemins d'exécution différents (ici le fait que certains calculent une bordure de masque et d'autres non), une divergence se produit, contraignant le matériel à désactiver certains threads, engendrant de la latence.

À l'inverse, l'implémentation `m5_static` permet d'esquiver ce problème. Comme les dimensions des tuiles, l'étendue de la marge et le nombre de threads cibles sont tous connus à l'avance, on peut deviner qu'exactement 3 chargements bruts suffiront pour rapatrier la mémoire utile à chaque thread. Les boucles sont intégralement supprimées et les branches aplaties par le compilateur. La suppression de cette surcharge de contrôle fluidifie l'exécution des *warp*, menant au gain observé.

5.2 — Exploration des tailles de tuiles : heatmap

Nous étudions ici l'impact des dimensions du groupe de travail (`tilew` et `tileh`) sur les temps d'exécution. Les figures suivantes (16 et 17) présentent les mesures relevées pour le noyau naïf (`ocl`) et pour notre version tuilée avec marge dynamique (`ocl_multi_m4_loop`).

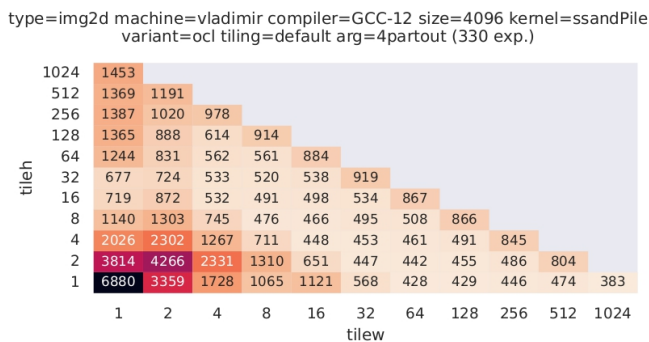


FIGURE 16 – Noyau naïf (`ocl`)

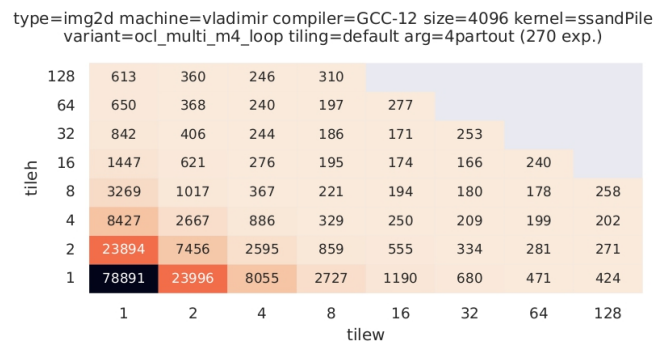


FIGURE 17 – Noyau `m4_loop`

L'évaluation comparative de ces deux heatmaps permet de mettre en lumière plusieurs phénomènes matériels. D'abord, et pour ces deux dernières, les blocs de base privilégiant la largeur (grand `tilew`) sont plus performants que ceux étirés en hauteur (grand `tileh`). Ce contraste vient de la coalescence mémoire. Lors de l'accès à la mémoire globale, le contrôleur regroupe les requêtes des threads d'un *warp* en transactions de 128 octets alignées lorsque les adresses sont contiguës. Avec un `tilew` élevé, les accès sont coalescés et efficaces. En revanche, lorsque `tilew` = 1, chaque thread accède à une ligne différente, ce qui entraîne des accès "espacés". Ces accès empêchent la coalescence et provoquent un grand nombre de transactions mémoire, réduisant fortement la bande passante effective.

De plus, les tuiles 1×1 affichent les pires chrono. Formater des groupes de travail contenant un unique thread gâche évidemment les ressources d'un SM car la structure interne synchronise le *warp* complet de 32 threads, dont 31 tourneront à vide. Le coût de lancement et de communication surpasse de très loin le temps d'exécution.

Enfin, les espaces vides en haut à droite des graphiques représentent les exécutions compromises. Sur OpenCL, le plafond classique total de threads par groupe de travail oscille historiquement autour de 1024. Cependant, la figure 17 montre davantage de configurations inéligibles. L'explication se trouve dans notre empreinte mémoire. Les marges requièrent l'allocation de deux tableaux locaux de dimensions $(\text{tilew} + 8) \times (\text{tileh} + 8)$. Les grandes tuiles en `m4_loop` surpassent la limite locale fixe allouée par le matériel, ce que le compilateur rejette par sécurité en amont.

5.3 — Impact de `BATCH_SIZE` sur la détection de terminaison

Cette expérience s'intéresse à l'impact de `BATCH_SIZE` (désigné par le paramètre `config` sur l'axe des abscisses) lors de l'exécution du noyau `ocl_multi_m4_loop`. Ce paramètre définit la fréquence de détection de terminaison : il indique le nombre de lancements de noyaux (chaque noyau effectuant 4 itérations internes) avant que l'hôte ne

vérifie de manière bloquante le drapeau de modification (`changed`) via `clEnqueueReadBuffer`. Pour cette expérience, le nombre maximal d'itérations est fixé à 1000, un nombre divisible par 4.

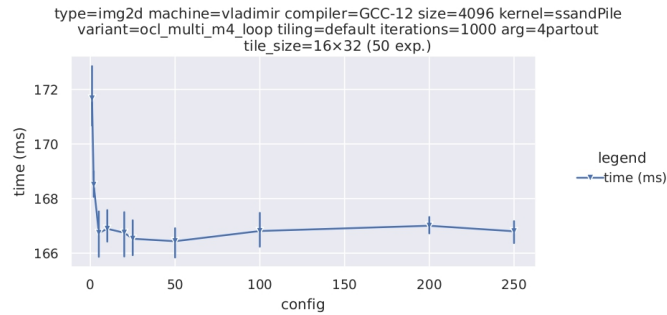


FIGURE 18 – Impact du paramètre de synchronisation `BATCH_SIZE` sur le temps d'exécution (noyau `m4_loop`, 4096×4096 , 1000 itérations).

Le comportement asymptotique du graphique est très instructif sur la latence matérielle. À `config = 1`, le temps culmine à ≈ 173 ms. Avec ce paramètre, on force une lecture bloquante du drapeau depuis le CPU toutes les 4 itérations internes. Pour atteindre les 1000 itérations, on provoque environ 250 synchronisations via le bus PCIe. Un simple calcul relie cette observation au hardware : le temps se stabilise ensuite vers 166 ms, la différence de ≈ 7 ms correspond donc à nos 250 appels de transfert. Cela donne une latence moyenne d'environ $28 \mu\text{s}$ par synchronisation `clEnqueueReadBuffer`, ce qui donne à peu près l'ordre de grandeur théorique de l'over-head PCIe pour un transfert CPU-GPU bloquant.

Dès lors que `config` dépasse 5 ou 10, le nombre de vérifications chute drastiquement (moins de 50 synchronisations totales). Le coût réseau devient quasiment invisible, et le temps s'aplatit autour de 166 ms.

Il faut noter cependant que, comme l'expérience est ici arrêtée à 1000 itérations, nous ne voyons pas la pénalité typique d'une vérification trop rare. Dans une simulation à convergence inconnue, définir un très grand `BATCH_SIZE` forcerait la grille à calculer inutilement des centaines d'états si elle se stabilisait dès la 5ème itération. Ici, l'aplatissement prouve que tant qu'on limite intelligemment la barrière CPU-GPU, le plafond de performance de la carte est atteint.

6. — Conclusion et perspectives

Ce projet met en parallèle deux stratégies d'optimisation. Sur CPU, la vectorisation AVX2 associée à une évaluation paresseuse est très efficace pour les grilles creuses, mais elle reste contrainte par la taille fixe des registres SIMD. Sur GPU, l'exploitation de la mémoire locale et le batching permettent un bon débit sur des données denses, au prix d'un sur-calcul et d'une faible adaptabilité aux structures éparpillées. Une amélioration consisterait à introduire un mécanisme paresseux adapté au modèle SIMT afin de limiter le gaspillage. Enfin, une approche hybride combinant CPU et GPU permettrait d'exploiter leurs complémentarités, en confiant les zones denses au GPU et les régions creuses au CPU.

Annexes

A — Kernels OpenCL

Listing 3 – Kernel OpenCL ocl_multi_m4_loop : chargement dynamique

```
1 __kernel void ssandPile_ocl_multi_m4_loop(__global unsigned *in,
2                                     __global unsigned *out, __global int *changed) {
3     #define MARGIN 4
4     #define TRUE_W (TILE_W + 2 * MARGIN)
5     #define IN_W TRUE_W
6     #define IN_H (TILE_H + 2 * MARGIN)
7     __local unsigned tile0[IN_H * IN_W], tile1[IN_H * IN_W];
8     __local int local_changed;
9
10    int lx = get_local_id(0), ly = get_local_id(1);
11    int tid = ly * TILE_W + lx;
12    int wg_size = TILE_W * TILE_H;
13    int gx = get_global_id(0), gy = get_global_id(1);
14    int base_gx = get_group_id(0) * TILE_W, base_gy = get_group_id(1) * TILE_H;
15    int total_cells = IN_H * IN_W;
16
17    if (tid == 0) local_changed = 0;
18
19    // --- Chargement cooperatif dynamique ---
20    for (int i = tid; i < total_cells; i += wg_size) {
21        int tx = i % IN_W, ty = i / IN_W;
22        int g_x = clamp(base_gx - MARGIN + tx, 0, DIM - 1);
23        int g_y = clamp(base_gy - MARGIN + ty, 0, DIM - 1);
24        tile0[i] = in[g_y * DIM + g_x];
25    }
26    barrier(CLK_LOCAL_MEM_FENCE);
27
28    // --- Iterations en memoire partagee ---
29    for (int i = tid; i < total_cells; i += wg_size) {
30        int tx = i % IN_W, ty = i / IN_W;
31        if (tx >= 1 && tx < IN_W - 1 && ty >= 1 && ty < IN_H - 1) {
32            unsigned c = tile0[i];
33            tile1[i] = (c & 3) + (tile0[i - IN_W] >> 2) + (tile0[i + IN_W] >> 2)
34                      + (tile0[i - 1] >> 2) + (tile0[i + 1] >> 2);
35        }
36    }
37    barrier(CLK_LOCAL_MEM_FENCE);
38    // (Repetition pour les passes internes restantes... vers tile0 puis tile1)
39
40    // --- Ecriture finale ---
41    int out_i = (ly + MARGIN) * IN_W + (lx + MARGIN);
42    unsigned center = tile1[out_i]; // Valeur finale de la cellule du thread
43    unsigned res = (center & 3) + (tile1[out_i - IN_W] >> 2) + (tile1[out_i + IN_W] >> 2)
44                  + (tile1[out_i - 1] >> 2) + (tile1[out_i + 1] >> 2);
45
46    if (gx > 0 && gx < DIM - 1 && gy > 0 && gy < DIM - 1) {
47        out[gy * DIM + gx] = res;
48        if (res != center) atomic_or(&local_changed, 1);
49    }
50    barrier(CLK_LOCAL_MEM_FENCE);
51    if (tid == 0 && local_changed) atomic_or(changed, 1);
52 }
```

Listing 4 – Kernel OpenCL ocl_multi_m5_static : chargement statique

```
1 // Ce noyau predit la charge sans boucle modulo-dependante
2 __kernel void ssandPile_ocl_multi_m5_static(__global unsigned *in,
3                                     __global unsigned *out, __global int *changed) {
```

```

4  #define MARGIN4 5
5  // ... [Configuration locale identique] ...
6
7  int i1 = tid, i2 = tid + wg_size, i3 = tid + 2 * wg_size; // Projection 3-pass statique
8  int tx1 = i1 % IN_W4, ty1 = i1 / IN_W4;
9  // etc... precalcul direct des coordonnees de tuile limitant les calculs arithmetiques
10
11 // 1. STATIC LOAD (3 affectations explicites, pre-unrolled, pas de boucle de garde
    continue)
12 {
13     tile0[i1] = in[clamp(base_gy - MARGIN4 + ty1, 0, DIM - 1) * DIM + clamp(base_gx -
        MARGIN4 + tx1, 0, DIM - 1)];
14     if (i2 < total_cells)
15         tile0[i2] = in[clamp(base_gy - MARGIN4 + ty2, 0, DIM - 1) * DIM + clamp(...tx2...
            )];
16     if (i3 < total_cells)
17         tile0[i3] = in[clamp... ];
18 }
19 barrier(CLK_LOCAL_MEM_FENCE);
20 }

```

B — Fonction host (gestion BATCH_SIZE)

Listing 5 – Bouclage hôte OpenCL pour le noyau tuilé

```

1  unsigned ssandPile_compute_ocl_multi_m4_loop(unsigned nb_iter) {
2      // Setup buffers, args...
3      const int zero = 0;
4      unsigned total_it = 0;
5
6      for (unsigned it = 1; it <= nb_iter; it += gpu_batch_size * 4) {
7          // 1. Reinitialisation locale du buffer de stabilisation global
8          err = clEnqueueFillBuffer(ocl_queue(0), ocl_changed_buffer, &zero,
9                                   sizeof(int), 0, sizeof(int), 0, NULL, NULL);
10
11         unsigned max_k = min(gpu_batch_size, (nb_iter - it + 1 + 3) / 4);
12         if (max_k == 0) break;
13
14         // 2. Lancement par vagues (gpu_batch_size fois)
15         for (unsigned k = 0; k < max_k; k++) {
16             total_it += 4; // Un lancement de noyau = 4 iterations passees
17             clEnqueueNDRangeKernel(ocl_queue(0), ocl_compute_kernel(0), 2, NULL, global, local, 0,
18                                    NULL, NULL);
19
20             // Echange asynchrone des buffers In/Out a la volée
21             cl_mem tmp = ocl_cur_buffer(0);
22             ocl_cur_buffer(0) = ocl_next_buffer(0);
23             ocl_next_buffer(0) = tmp;
24             clSetKernelArg(ocl_compute_kernel(0), 0, sizeof(cl_mem), &ocl_cur_buffer(0));
25             clSetKernelArg(ocl_compute_kernel(0), 1, sizeof(cl_mem), &ocl_next_buffer(0));
26         }
27
28         // 3. Telechargement bloquant du flag de stabilite (Cree l'overhead PCIe observe)
29         int changed;
30         clEnqueueReadBuffer(ocl_queue(0), ocl_changed_buffer, CL_TRUE, 0, sizeof(int), &changed,
31                             0, NULL, NULL);
32
33         if (changed == 0) { // Detection conditionnee par l'aggregation GPU
34             clFinish(ocl_queue(0));
35             return total_it;
36         }
37     }
38     return 0;
39 }

```

C — Vectorisation synchrone et asynchrone

Listing 6 – Fonction de tuile AVX sandPile synchrone en parcours horizontal

```
1 int ssandPile_do_tile_avx (int x, int y, int width, int height)
2 {
3     int left_tile  = x == 1;
4     int right_tile = x + width == DIM - 1;
5     width += left_tile + right_tile; // add missing width
6     x -= left_tile; // set x to 0 if it was added 1 on side
7
8     int diff = 0;
9
10    TYPE *restrict in_cell  = table_cell (TABLE, in, y, x);
11    TYPE *restrict out_cell = table_cell (TABLE, out, y, x);
12    const register __m256i MASK_ZERO = _mm256_set1_epi32 (0);
13    const register __m256i MASK_MOD_4 = _mm256_set1_epi32 (3);
14
15    for (int i = 0; i < height; i++) {
16        int is_border_up    = (i == 0);
17        int is_border_down  = (i == (height - 1));
18
19        TYPE *restrict up_row    = in_cell - DIM;
20        TYPE *restrict center_row = in_cell;
21        TYPE *restrict down_row  = in_cell + DIM;
22
23        register __m256i center_vec;
24        center_vec = _mm256_load_si256 ((__m256i *)center_row);
25
26        for (int j = 0; j < width; j += AVX_VEC_SIZE_INT) {
27            int is_border_left  = (j == 0);
28            int is_border_right = (j == (width - AVX_VEC_SIZE_INT));
29
30            //side tile handles: do left and right only if not on a
31            //left/right tile and not on a left/right border
32            int do_left        = !(is_border_left & left_tile);
33            int do_right       = !(is_border_right & right_tile);
34
35            // set_epi32 has a weird reversed order
36            __m256i mask = _mm256_set_epi32 (-1 * do_right, -1, -1, -1, -1, -1, -1,
37                                             -1 * do_left);
38
39            //load up/down
40            __m256i up_vec  = _mm256_load_si256 ((__m256i *) (up_row + j));
41            __m256i down_vec = _mm256_load_si256 ((__m256i *) (down_row + j));
42
43            //load offseted vectors
44            __m256i left_added_vec =
45                _mm256_loadu_si256 ((__m256i *) (center_row + j + 1));
46            __m256i right_added_vec =
47                _mm256_loadu_si256 ((__m256i *) (center_row + j - 1));
48
49            __m256i res = _mm256_add_epi32 (_mm256_srli_epi32 (left_added_vec, 2),
50                                           _mm256_srli_epi32 (right_added_vec, 2));
51            res = _mm256_add_epi32 (res, _mm256_and_si256 (center_vec, MASK_MOD_4));
52
53            __m256i tops = _mm256_add_epi32 (_mm256_srli_epi32 (down_vec, 2),
54                                           _mm256_srli_epi32 (up_vec, 2));
55            res = _mm256_add_epi32 (res, tops);
56
57            // we can zero out incorrect cells, or do a maskstore
58            res = _mm256_blendv_epi8 (MASK_ZERO, res, mask);
59            _mm256_store_si256 ((__m256i *) (out_cell + j), res);
60        }
```



```

61     __m256 cast                = (__m256)_mm256_cmpeq_epi32 (res, center_vec);
62     unsigned char change_vec = ~(unsigned char)_mm256_movemask_ps (cast);
63
64     char change_left  = change_vec & 1;
65     char change_right = (change_vec >> (AVX_VEC_SIZE_INT - 1)) & 1;
66
67     int self_flag = SANDPILE_BORDER_SET_SELF (change_vec != 0);
68     int up_flag   = SANDPILE_BORDER_SET_UP   (self_flag, is_border_up);
69     int down_flag = SANDPILE_BORDER_SET_DOWN (self_flag, is_border_down);
70     int left_flag  = SANDPILE_BORDER_SET_LEFT (change_left, is_border_left);
71     int right_flag =
72         SANDPILE_BORDER_SET_RIGHT (change_right, is_border_right);
73
74     //Border handle
75     diff |= self_flag | up_flag | down_flag | left_flag | right_flag;
76 }
77
78 in_cell += DIM;
79 out_cell += DIM;
80 }
81
82 return diff;
83 }

```

Cette version de fonction de tuile correspond à la version en parcours horizontal. Elle contient un traitement des tuiles de bord et une détection fine des changements sur les côtés (en suivant les spécifications que nous avons définis dans nos précédents travaux). Ici, le mask pour le traitement des tuiles de bord est fait via un blend avant le store du vecteur.

Listing 7 – Fonction de tuile AVX optimisée sandPile synchrone

```

1
2 int ssandPile_do_tile_opt_avx (int x, int y, int width, int height)
3 {
4     const __m256i m256_3 = _mm256_set1_epi32 (3);
5     const int offset     = -(height * DIM) + AVX_VEC_SIZE_INT;
6     int diff              = 0;
7
8     int left_tile  = x == 1;
9     int right_tile = x + width == DIM - 1;
10    width += left_tile + right_tile;
11    x -= left_tile;
12
13    TYPE *in_cell = table_cell (TABLE, in, y, x);
14    TYPE *out_cell = table_cell (TABLE, out, y, x);
15
16    for (int j = 0; j < width; j += AVX_VEC_SIZE_INT) {
17        int is_border_left  = (j == 0);
18        int is_border_right = (j == (width - AVX_VEC_SIZE_INT));
19
20        //side tile handles: do left/right only if not on a
21        //left/right tile and not on a left/right border
22        int do_left      = -1 * !(is_border_left & left_tile);
23        int do_right     = -1 * !(is_border_right & right_tile);
24        __m256i mask = _mm256_set_epi32 (do_right, -1, -1, -1, -1, -1, do_left);
25
26        register __m256i up_vec = _mm256_load_si256 ((__m256i *) (in_cell - DIM));
27        register __m256i center_vec = _mm256_load_si256 ((__m256i *) (in_cell));
28        register __m256i down_vec;
29
30        SANDPILE_UNROLL_LOOP (4)
31        for (int i = 0; i < height; i++) {
32            int is_border_up = (i == 0);
33            int is_border_down = (i == (height - 1));
34            down_vec = _mm256_load_si256 ((__m256i *) (in_cell + DIM));

```

```

35
36 __m256i res = _mm256_and_si256 (center_vec, m256_3);
37
38 //load offseted vectors
39 __m256i left_vec = _mm256_loadu_si256 ((__m256i *) (in_cell - 1));
40 __m256i right_vec = _mm256_loadu_si256 ((__m256i *) (in_cell + 1));
41
42 __m256i tops = _mm256_add_epi32 (_mm256_srli_epi32 (up_vec, 2),
43                                 _mm256_srli_epi32 (down_vec, 2));
44
45 __m256i sides = _mm256_add_epi32 (_mm256_srli_epi32 (left_vec, 2),
46                                 _mm256_srli_epi32 (right_vec, 2));
47
48 sides = _mm256_add_epi32 (tops, sides);
49 res = _mm256_add_epi32 (res, sides);
50
51 // we can zero out incorrect cells, or do a maskstore
52 res = _mm256_and_si256 (res, mask);
53 _mm256_store_si256 ((__m256i *) (out_cell), res);
54
55 __m256 cast = (__m256) _mm256_cmpeq_epi32 (res, center_vec);
56 unsigned char change_vec = ~(unsigned char) _mm256_movemask_ps (cast);
57
58 char change_right = (change_vec >> (AVX_VEC_SIZE_INT - 1)) & 1;
59 char change_left = change_vec & 1;
60
61 int self_flag = SANDPILE_BORDER_SET_SELF (change_vec != 0);
62 int up_flag = SANDPILE_BORDER_SET_UP (self_flag, is_border_up);
63 int down_flag = SANDPILE_BORDER_SET_DOWN (self_flag, is_border_down);
64 int left_flag = SANDPILE_BORDER_SET_LEFT (change_left, is_border_left);
65 int right_flag =
66     SANDPILE_BORDER_SET_RIGHT (change_right, is_border_right);
67
68 //Border handle
69 diff |= self_flag | up_flag | down_flag | left_flag | right_flag;
70
71 //shift up/center vec
72 up_vec = center_vec;
73 center_vec = down_vec;
74
75 in_cell += DIM;
76 out_cell += DIM;
77 }
78 in_cell += offset;
79 out_cell += offset;
80 }
81
82 return diff;
83 }

```

Cette version de notre fonction de tuile correspond à la version AVX en parcours vertical. Cette version est plus optimisée que la précédente. De nouveau, elle contient un traitement fin des changements au bord et un traitement des tuiles de bords. Ce traitement des bords est fait via un and logique avec le mask. Il faudrait comparer les performances entre les différentes façons de masquer les cellules (un maskstore serait sûrement mieux).

Listing 8 – Fonction de tuile AVX sandPile asynchrone

```

1
2
3 int asandPile_do_tile_avx (int x, int y, int width, int height)
4 {
5     int up_tile = y == 1;
6     int down_tile = y + height == DIM - 1;
7     int left_tile = x == 1;
8     int right_tile = x + width == DIM - 1;

```

```

9
10 // reset potentially added width, height, x,y
11 width += left_tile + right_tile;
12 height += up_tile + down_tile;
13 x -= left_tile;
14 y -= up_tile;
15
16 TYPE *restrict table_ptr = atable_cell (TABLE, y, x);
17 int diff = 0;
18
19 /* Shuf scheme that allows all the lane crossing changes to be done at
20  * once */
21 const int NOP = 0; /* Dummy, only look at var name, not value */
22 const __m256i VEC_SHUFFLE_IDX =
23     _mm256_setr_epi32 (NOP, NOP, NOP, 4, 3, NOP, NOP, NOP);
24
25 const __m256i VEC_ZERO = _mm256_setzero_si256 ();
26 const __m256i VEC_MOD_FOUR = _mm256_set1_epi32 (3);
27 const int SUBTILE_SIZE = 8;
28
29 __m256i subtile [SUBTILE_SIZE + 2];
30 __m256i subtile_leftover [SUBTILE_SIZE];
31
32 const int MAX_INNER_PASSES = 4;
33 const int MAX_SUBTILE_PASSES = 4;
34
35 int ut = 0;
36 int iter_change = 1;
37 while (iter_change && (ut < MAX_INNER_PASSES)) {
38     ut++;
39     iter_change = 0;
40     for (int ut = 0; ut < 2; ut++) {
41         for (int j = 0; j < width; j += AVX_VEC_SIZE_INT) {
42             int is_border_left = (j == 0);
43             int is_border_right = (j == (width - AVX_VEC_SIZE_INT));
44
45             //side tile handles: do left/right only if not on a
46             //left/right tile and not on a left/right border
47             int do_left = !(is_border_left & left_tile);
48             int do_right = !(is_border_right & right_tile);
49
50             __m256i mask = _mm256_setr_epi32 (-1 * do_left, -1, -1, -1, -1, -1, -1,
51                 -1 * do_right);
52
53             for (int i = 0; i < height; i += SUBTILE_SIZE) {
54                 // subtile size for this iteration: used for top and bottom tiles
55                 const int use_subtile_size =
56                     SUBTILE_SIZE - 1 * ((up_tile * (i == 0)) +
57                         (down_tile * (i + SUBTILE_SIZE == height)));
58
59                 /* ==== PART 1: load ==== */
60                 TYPE *restrict p_cell =
61                     table_ptr + (i + (up_tile * (i == 0))) * DIM + j;
62
63                 for (int k = 0; k < use_subtile_size; k++) {
64                     subtile [k + 1] =
65                         _mm256_maskload_epi32 ((int *) (p_cell + DIM * k), mask);
66                     subtile_leftover [k] = VEC_ZERO;
67                 }
68
69                 // Up and down vectors (hold spilled values)
70                 subtile [0] = _mm256_load_si256 ((__m256i *) (p_cell - DIM));
71                 subtile [use_subtile_size + 1] =
72                     _mm256_load_si256 ((__m256i *) (p_cell + DIM * use_subtile_size));

```

```

73
74  /* ==== PART 2: compute ==== */
75
76  int it = 0;
77  int loc_change = 1;
78  while (loc_change && (it < MAX_SUBTILE_PASSES)) {
79      it++;
80      loc_change = 0;
81
82      for (int k = 0; k < use_subtile_size; k++) {
83          int idx = k + 1;
84          __m256i topple = _mm256_srli_epi32 (subtile [idx], 2);
85
86          // shifts between lanes
87          __m256i vec_permuted =
88              _mm256_permutevar8x32_epi32 (topple, VEC_SHUFFLE_IDX);
89
90          subtile [idx - 1] = _mm256_add_epi32 (subtile [idx - 1], topple);
91          subtile [idx + 1] = _mm256_add_epi32 (subtile [idx + 1], topple);
92
93          // shifts in lanes and mix with shuffled
94          __m256i right_topple = _mm256_blend_epi32 (
95              _mm256_bslli_epi128 (topple, 4), vec_permuted, 0b00010000);
96
97          __m256i left_topple = _mm256_blend_epi32 (
98              _mm256_bslli_epi128 (topple, sizeof (unsigned int)),
99              vec_permuted, 0b00001000);
100
101          // left/right topple:
102          // we abuse the fact that we wont use center values
103          // so we add everything.
104          subtile_leftover [k] =
105              _mm256_add_epi32 (subtile_leftover [k], topple);
106
107          __m256i res = _mm256_and_si256 (subtile [idx], VEC_MOD_FOUR);
108          __m256i total_topple =
109              _mm256_add_epi32 (right_topple, left_topple);
110          res = _mm256_add_epi32 (res, total_topple);
111          subtile [idx] = res;
112
113          __m256 cast = (__m256)_mm256_cmpeq_epi32 (topple, VEC_ZERO);
114          unsigned char change_vec =
115              ~(unsigned char)_mm256_movemask_ps (cast);
116
117          diff |= change_vec != 0;
118
119          loc_change |= diff;
120          iter_change |= diff;
121      }
122  }
123
124  /* ==== PART 3: store ==== */
125
126  for (int k = 0; k < use_subtile_size; k++) {
127      __m256i value = (subtile [k + 1]);
128      _mm256_store_si256 ((__m256i *) (p_cell + DIM * k), value);
129
130      int left = _mm256_extract_epi32 (subtile_leftover [k], 0);
131      int right = _mm256_extract_epi32 (subtile_leftover [k], 7);
132
133      *(p_cell + DIM * k - 1 * do_left) += left;
134      *(p_cell + DIM * k + AVX_VEC_SIZE_INT * do_right) += right;
135  }
136

```

```

137     __mm256_store_si256 ((__m256i *) (p_cell - DIM), subtile [0]);
138     __mm256_store_si256 ((__m256i *) (p_cell + DIM * use_subtile_size),
139                          subtile [use_subtile_size + 1]);
140 }
141 }
142 }
143 }
144
145 return diff;
146 }

```

Fonction d'exécution d'une tuile en AVX pour `asandPile`. Cette fonction gère le traitement des bords et n'est pas la plus optimisée. Elle utilise une sous-tuile pour exécuter ses itérations synchrones internes. Les valeurs pour la taille d'une sous-tuile, le nombre d'itérations de boucles et le nombre d'itérations dans la sous-tuiles, ne sont pas les plus optimisées. Cette fonction ayant été sujette a beaucoup de tests sur ces paramètres, nous les avons laissé tels quels. Pour les sous-tuiles, on considère deux vecteurs supplémentaires : le vecteur directement en haut et celui directement en bas, nous avons fait le choix de considérer des tailles de sous-tuiles en puissance de 2. Par conséquent, le traitement des tuiles de bords haut n'est pas trivial. La variable `VEC_SHUFFLE_IDX` permet de réaliser l'optimisation sur les shifts intra-lanes dont nous parlions plus tôt. La variable `use_subtile_size` permet de définir la vraie taille de sous-tuile lors de l'exécution d'une tuile de bords haut. L'itération à chaque boucle est décomposée en 3 parties : le load, le calcul, et le store. Seul le load est masqué, la raison étant que l'on peut store des valeurs arbitraires sur les bords tant que l'on ne s'en sert pas après, ce qui est vrai. Pour rappel, cette version obtient de moins bons résultats que la version sans traitement des bords (pour paramètres de tailles de tuiles internes et d'itérations similaires.). Enfin, le vecteur `subtile_leftover` maintient les valeurs éboulées de la gauche/droite pour chacun de nos vecteurs. Nous les gardons comme vecteurs pendant le traitement et les extrayons à la fin.

ERRATUM: En écrivant la phrase sur le masquage du load, je me suis rendu compte que cette façon de procéder pouvait considérer les cellules de bord comme des calculs ayant apporté des modifications. Cela forcerait potentiellement des calculs en plus ce qui pourrait expliquer nos différences de performances sur la version considérant les tuiles de bords. Plus de tests sont nécessaires pour confirmer cela.

Listing 9 – Fonction de tuile AVX défaut `sandPile` asynchrone

```

1
2 int asandPile_do_tile_avx_default (int x, int y, int width, int height)
3 {
4     if (width != TILE_W || height != TILE_H)
5         return asandPile_do_tile_opt (x, y, width, height);
6
7     TYPE *restrict table_ptr = atable_cell (TABLE, y, x);
8     int diff = 0;
9
10    /* Shuf scheme that allows all the lane crossing changes to be done at
11     * once */
12    const int NOP = 0; /* Dummy, only look at var name, not value */
13    const __m256i VEC_SHUFFLE_IDX =
14        __mm256_setr_epi32 (NOP, NOP, NOP, 4, 3, NOP, NOP, NOP);
15
16    const __m256i VEC_ZERO = __mm256_setzero_si256 ();
17    const __m256i VEC_MOD_FOUR = __mm256_set1_epi32 (3);
18    const int SUBTILE_SIZE = 4;
19
20    __m256i subtile [SUBTILE_SIZE + 2];
21    __m256i subtile_leftover [SUBTILE_SIZE];
22
23    const int MAX_INNER_PASSES = 1;
24    const int MAX_SUBTILE_PASSES = 2;
25
26    int ut = 0;
27    int iter_change = 1;
28    while ((ut < MAX_INNER_PASSES) & iter_change) {
29        ut++;
30        iter_change = 0;

```

```

31 for (int ut = 0; ut < 2; ut++) {
32     for (int j = 0; j < width; j += AVX_VEC_SIZE_INT) {
33
34         for (int i = 0; i < height; i += SUBTILE_SIZE) {
35
36             /* ==== PART 1: load ==== */
37             TYPE *restrict p_cell = table_ptr + i * DIM + j;
38
39             for (int k = 0; k < SUBTILE_SIZE; k++) {
40                 subtile [k + 1] = __mm256_load_si256 ((__m256i *) (p_cell + DIM * k));
41                 subtile_leftover [k] = VEC_ZERO;
42             }
43
44             subtile [0] = __mm256_load_si256 ((__m256i *) (p_cell - DIM));
45             subtile [SUBTILE_SIZE + 1] =
46                 __mm256_load_si256 ((__m256i *) (p_cell + DIM * SUBTILE_SIZE));
47
48             /* ==== PART 2: compute ==== */
49
50             int it = 0;
51             int loc_change = 1;
52             while ((it < MAX_SUBTILE_PASSES) & loc_change) {
53                 it++;
54                 loc_change = 0;
55
56                 for (int k = 0; k < SUBTILE_SIZE; k++) {
57                     int idx = k + 1;
58                     __m256i topple = __mm256_srli_epi32 (subtile [idx], 2);
59
60                     __m256i vec_permuted =
61                         __mm256_permutevar8x32_epi32 (topple, VEC_SHUFFLE_IDX);
62
63                     subtile [idx - 1] = __mm256_add_epi32 (subtile [idx - 1], topple);
64                     subtile [idx + 1] = __mm256_add_epi32 (subtile [idx + 1], topple);
65
66                     __m256i right_topple = __mm256_blend_epi32 (
67                         __mm256_bslli_epi128 (topple, 4), vec_permuted, 0b00010000);
68
69                     __m256i left_topple = __mm256_blend_epi32 (
70                         __mm256_bsrli_epi128 (topple, sizeof (unsigned int)),
71                         vec_permuted, 0b00001000);
72
73                     subtile_leftover [k] =
74                         __mm256_add_epi32 (subtile_leftover [k], topple);
75
76                     __m256i res = __mm256_and_si256 (subtile [idx], VEC_MOD_FOUR);
77                     __m256i total_topple =
78                         __mm256_add_epi32 (right_topple, left_topple);
79                     res = __mm256_add_epi32 (res, total_topple);
80                     subtile [idx] = res;
81
82                     __m256 cast = (__m256) __mm256_cmpeq_epi32 (topple, VEC_ZERO);
83                     unsigned char change_vec =
84                         ~(unsigned char) __mm256_movemask_ps (cast);
85
86                     diff |= change_vec != 0;
87                     loc_change |= diff;
88                     iter_change |= diff;
89                 }
90             }
91
92             /* ==== PART 3: store ==== */
93
94             for (int k = 0; k < SUBTILE_SIZE; k++) {

```

```

95     __m256i value = (subtile [k + 1]);
96     __mm256_store_si256 ((__m256i *) (p_cell + DIM * k), value);
97
98     int left  = _mm256_extract_epi32 (subtile_leftover [k], 0);
99     int right = _mm256_extract_epi32 (subtile_leftover [k], 7);
100
101     *(p_cell + DIM * k - 1) += left;
102     *(p_cell + DIM * k + AVX_VEC_SIZE_INT) += right;
103 }
104
105 __mm256_store_si256 ((__m256i *) (p_cell - DIM), subtile [0]);
106 __mm256_store_si256 ((__m256i *) (p_cell + DIM * SUBTILE_SIZE),
107                     subtile [SUBTILE_SIZE + 1]);
108 }
109 }
110 }
111 }
112
113 return diff;
114 }

```

Cette version de tuilage AVX est en tout point similaire à la précédente exceptée qu'elle ne gère pas les tuiles de bords, et que les paramètres de taille de tuile et d'itérations internes sont différents (ici ceux obtenus après tous nos tests). En pratique, il s'agit de celle utilisée.

Par ailleurs, il existe des versions "_border" de ces fonctions permettant de détecter les changements fin des bords. Nous ne les incluons pas car ces changements sont strictement identiques à ceux des fonctions du noyau synchrone.

Listing 10 – Fonction de regroupement des Bitset AVX

```

1 static inline int tileset_merge_avx (tileset_t tile, tileset_t *restrict others,
2                                     unsigned nb_others)
3 {
4
5     __m256i VEC_ZERO = __mm256_setzero_si256();
6     int diff = 0;
7
8     for (unsigned j = 0; j < TOTAL_NB_SETS; j += BITSET_AVX_SIZE) {
9         __m256i *tile_at = (__m256i *) (&tileset_at (tile, j));
10        __m256i t = VEC_ZERO;
11
12        SANDPILE_UNROLL_LOOP (4)
13        for (unsigned i = 0; i < nb_others; i++) {
14            __m256i *other_at = (__m256i *) (&tileset_at (others[i], j));
15            __m256i other = _mm256_load_si256 (other_at);
16
17            t = _mm256_or_si256 (t, other);
18        }
19
20        __m256 cast = (__m256) _mm256_cmpeq_epi32 (t, VEC_ZERO);
21        __mm256_store_si256 (tile_at, t);
22
23        unsigned char change_vec = ~(unsigned char) _mm256_movemask_ps (cast);
24        diff |= change_vec != 0;
25    }
26
27    return diff;
28 }

```

Pour le rassemblement des bitset, nous avons choisis un parcours qui semble au contrepied de ce qui pourrait être la meilleure façon. Nous parcourons chaque vecteur de K éléments en passant d'abord par chaque autre bitset. En réalité, cela nous permet de limiter le nombre de load (puisque nous n'avons pas besoin de load la valeur originale dans le set). En pratique, ce parcours est meilleur que l'autre sens.

D — Informations complémentaires

Description détaillée des caches du CPU

La récupération de certaines informations CPU se sont faites au moyen des informations exposées par le noyau linux sous le répertoire `/sys/devices/system/cpu/cpu*/cache/index*`, où les `''` peut-être remplacés pour sélectionner le numéro du coeur et le niveau de cache respectivement. En l'occurrence, les tailles des lignes de cache sont trouvées grâce aux fichiers `coherency_line_size`, l'associativité grâce au fichier `ways_of_associativity`, nous ne détaillons pas tous les fichiers, mais toutes les informations données ensuite sont trouvable dans ces répertoires.

Nous travaillons donc avec une taille de ligne de cache de 512 bits pour tous les caches. Les caches L1 des coeurs peuvent tenir 64 sets, les P-cores sont à 12 way (12 lignes de cache par sets), les E-cores étant à 8 way. Tous les caches L2 possède 4096 sets à 12 way pour les P-cores contre 16 way pour les E-cores, mais ces derniers le partage entre eux. Enfin, Le cache L3 partagé entre tous les coeurs possède 49152 sets et est à 12 way

Comparaison des cache miss du traitement horizontal vs vertical pour ssandpile AVX

Lors du traitement AVX, le traitement horizontal déclenchera forcément 1 cache miss par traitement. On peut supposer que nos données sont alignées au début d'une ligne de cache ce qui est garanti par l'allocation mémoire avec `mmap` (alignée sur une page, donc sur une ligne de cache), et reste vrai pour le second tableau pour un problème de taille ≥ 16 . En effet, le premier vecteur traité (256 bits) sur 512 bits déclenchera un cache miss à gauche, le deuxième, un cache miss à droite. Pour le traitement vertical, lors du traitement des premiers 256 bits de chaque ligne de cache, la même chose est vraie, mais on rajoute cependant le cache miss de la ligne suivante en plus. Hors, si l'on considère uniquement le cache L1 (cache à 12 ou 8 way selon le coeur) et si l'on considère des tailles de tuiles effectives pour nos cas ≤ 8 , les lignes à moitié utilisées seront toujours présentes dans le cache. Sous cette hypothèse, le traitement de la deuxième partie de la ligne de cache verticalement se fera sans cache miss. Bien que ces hypothèses ne soient pas garanties, si elles le sont, le nombre de cache miss du traitement vertical et horizontal sera strictement équivalent. Cependant, on peut remarquer qu'une taille de tuile de 8 en vertical est assez basse et pourrait au contraire faire défaut aux performances.

Coût du regroupement des bitsets selon le nombre de threads.

Pour rappel, notre structure lazy bitset est utilisée dans un programme parallélisé en donnant une copie locale à chaque thread. Chaque thread marque alors les changements dans son propre bitset, et une passe de regroupement est faite à la fin. Plus le nombre de threads et d'entiers dans notre set seront grands, plus cette passe sera longue. Nous proposons alors un moyen de rendre constant le nombre de bitset peu importe combien de threads sont utilisés.

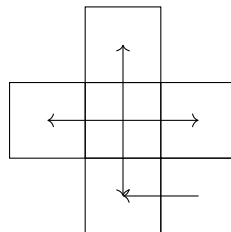


FIGURE 19 – Marquage d'une tuile par rapport aux autres tuiles.

On part de l'observation suivante, une tuile ne peut marquer que elle-même où ses voisins directs 19. Un conflit apparaît pour une tuile uniquement si deux tuiles la marquent en même temps. Dans notre exemple, cela revient à avoir deux directions pointant vers la même case. On représente alors une grille où l'on se concentre sur une seule des directions dans laquelle une tuile marque.

Dans une telle grille 20, peu importe comment des threads effectuent le marquage, il n'y a aucun conflit. Autrement dit, l'on peut réduire le problème de la concurrence pour le marquage dans notre set au problème : comment faire en sorte que dans un marquage où l'on considère les directions, aucune direction se croise. Pour cela, il suffit simplement de créer un set pour chaque direction de marquage. Ici, nous en avons 5, haut, bas, gauche, droite et "soi-même". Ainsi, peu importe le nombre de threads utilisé, un marquage à 5 sets où chaque thread marque ses changements à gauche dans le set 0, à droite dans le set 1, et ainsi de suite ne contient aucun conflit de marquage. Nous avons donc rendu constant notre nombre de sets à 5. Reste que ce changement a un surcoût : on augmente alors le nombre de faux partages et on espace la mémoire avec laquelle les threads travaillent, il convient donc que notre première implémentation en reste intéressante, il faudra tester à quel nombre de threads cette implémentation en devient meilleure. Nous n'avons pas implémenté cette structure étant donné que les problèmes sont assez petits (1024 et 2048). Il est bon de rappeler que notre implémentations utilise des entiers représentant plusieurs tuiles. Hors, chaque threads effectuent des "entiers" de tuiles, on peut remarquer qu'alors, ce marquage est toujours possible.

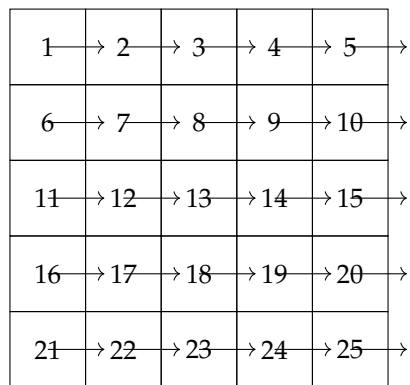


FIGURE 20 – Exemple d'un marquage à droite dans une grille de tuiles.